



UNIVERSITY OF TIRANA
FACULTY OF NATURAL SCIENCES

Project Documentation: Smart Contracts and Their Vulnerabilities

Blockchain for Distributed Applications

Project Type: Research & Practical Implementation

Submitted by : Dora Hoxha

TABLE OF CONTENTS

1. Abstract
2. Introduction

- 3. Theoretical Background
 - 3.1 Blockchain and Smart Contracts
 - 3.2 Immutability and the Security Paradox
 - 3.3 Common Smart Contract Vulnerabilities
- 4. Case Study: The DAO Attack
 - 4.1 Background and Context
 - 4.2 Technical Cause of the Exploit
 - 4.3 Financial and Systemic Impact
- 5. Research Methodology
- 6. Practical Implementation
 - 6.1 Development Environment
 - 6.2 Contract Design Overview
 - 6.3 Compilation and Deployment Process
 - 6.4 Transaction Verification and Testing
- 7. Security Evaluation and Efficiency Analysis
- 8. Discussion: Linking Theory with Practice
- 9. Conclusion and Lessons Learned

1. ABSTRACT

This project provides an in-depth academic and practical analysis of smart contract security within blockchain-based distributed systems. The main objective is to examine how theoretical vulnerabilities, specifically reentrancy attacks, manifest in real-world smart contracts and how secure development practices can effectively mitigate these risks.

The research begins with a comprehensive theoretical foundation, exploring blockchain immutability, the Ethereum Virtual Machine (EVM), and common vulnerability classes. This theoretical analysis is then linked to a historical case study of the 2016 DAO attack, which represents one of the most significant financial losses caused by a smart contract vulnerability.

To bridge theory with practice, the project implements two smart contracts: one intentionally vulnerable and one secured using industry best practices. These contracts are compiled, deployed, and verified in a controlled testing environment using Remix IDE, MetaMask, and Etherscan. The results highlight the importance of secure coding patterns, transparent verification, and disciplined deployment workflows.

2. INTRODUCTION

Blockchain technology has transformed the way trust is established in digital systems by enabling decentralized, tamper-resistant ledgers. Smart contracts play a central role in this transformation by allowing predefined rules and financial logic to execute automatically without intermediaries. These characteristics have made smart contracts a foundational element of decentralized finance (DeFi), decentralized governance, and digital asset management.

However, the same properties that make smart contracts powerful also introduce critical security

challenges. Once deployed, smart contracts become immutable, meaning that any vulnerability embedded in the code cannot be easily corrected. As a result, logical errors can evolve into permanent financial risks. This project investigates these challenges by combining academic research with a practical implementation that demonstrates how vulnerabilities arise and how they can be prevented through proper design and verification.

3. THEORETICAL BACKGROUND

3.1 Blockchain and Smart Contracts

Smart contracts are self-executing programs running on the Ethereum Virtual Machine (EVM). Their deterministic execution ensures transparency and predictability, but also means that any logical error will always be executed exactly as written.

3.2 Immutability and the Security Paradox

Immutability is one of blockchain's strongest guarantees, yet it creates a security paradox. Traditional software can be patched after deployment, while smart contracts cannot. This elevates the importance of secure design, testing, and verification before deployment.

3.3 Common Smart Contract Vulnerabilities

The research phase examined common vulnerabilities, including:

- Reentrancy attacks
- Integer overflow and underflow
- Improper access control
- Unchecked external calls

Among these, reentrancy was selected as the main focus due to its historical relevance and continued presence in modern DeFi exploits.

4. CASE STUDY: THE DAO ATTACK

4.1 Background and Context

The DAO (Decentralized Autonomous Organization) was an early Ethereum-based investment fund that raised approximately \$150 million. It represented one of the first large-scale financial smart contract systems.

4.2 Technical Cause of the Exploit

The vulnerability originated from a reentrancy flaw, where Ether was transferred before internal balances were updated. This allowed an attacker to repeatedly withdraw funds within a single transaction.

4.3 Financial and Systemic Impact

The attack resulted in losses of approximately \$60 million and led to a controversial hard fork of the Ethereum network. This event demonstrated that smart contract vulnerabilities are not theoretical risks, but real threats with global financial consequences.

5. RESEARCH METHODOLOGY

The project follows a theory-first approach supported by practical verification. The methodology includes:

- Literature review of smart contract vulnerabilities
- Analysis of historical attacks
- Implementation of minimal smart contracts
- Compilation, deployment, and transaction verification

6. PRACTICAL IMPLEMENTATION

6.1 Development Environment

- Remix IDE for coding and compilation
- Remix VM / Test Network for deployment
- MetaMask for transaction signing and interaction
- Etherscan for transaction and contract verification

6.2 Contract Design Overview

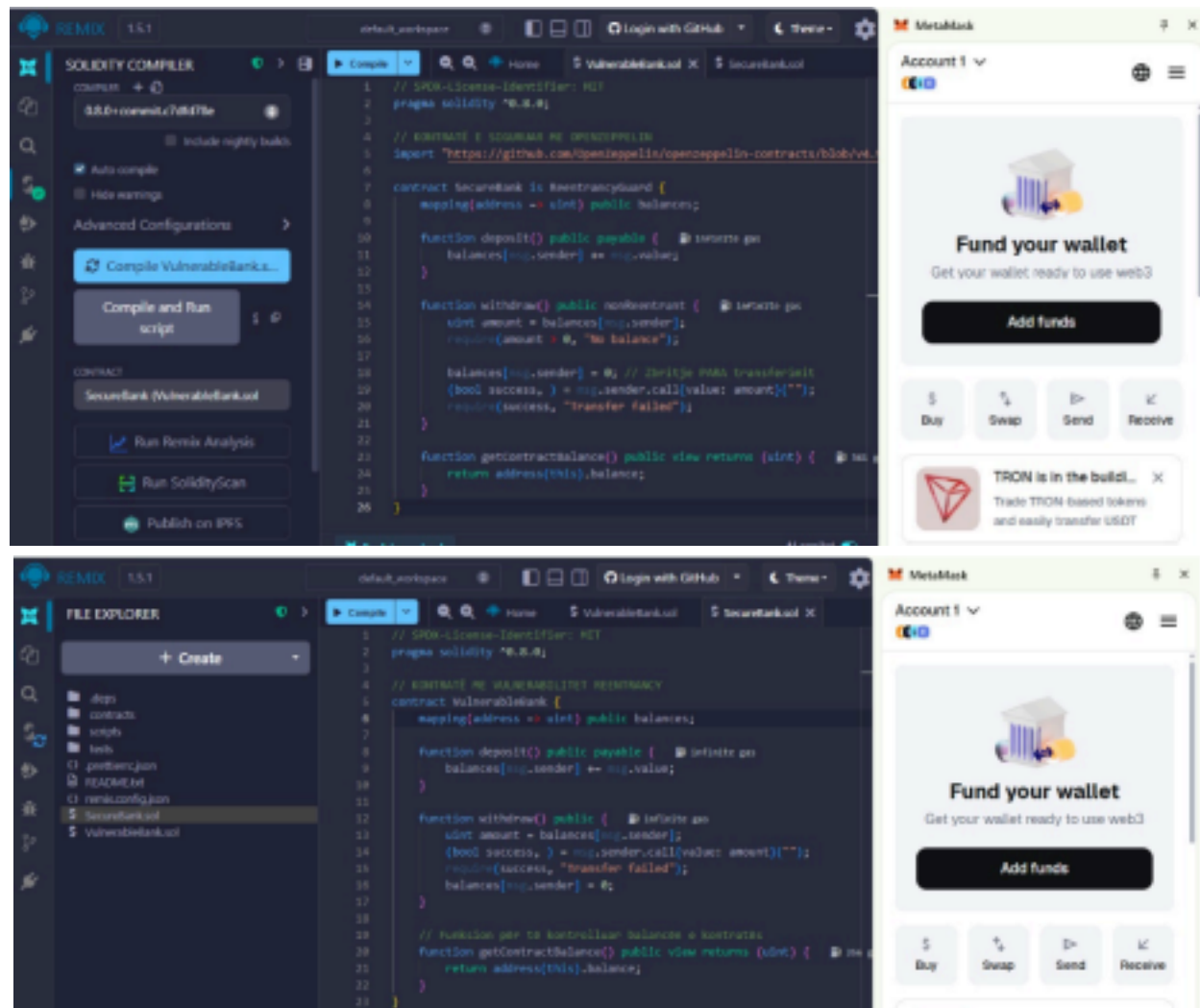
Two smart contracts were implemented:

1. A vulnerable contract demonstrating reentrancy risk
2. A secure contract applying best practices such as the Check-Effects-Interactions (CEI) pattern and reentrancy guards

This comparative design highlights how small changes in logic significantly impact security.

6.3 Compilation and Deployment Process

The contracts were compiled using Remix IDE to ensure syntactic correctness and compatibility with the EVM. Deployment was performed in a controlled test environment, simulating real blockchain conditions where code immutability applies.



Compilation and Structural Verification Process

In this case both contracts were:

- Successfully compiled in Remix IDE (Solidity v0.8.0+)
- Deployed conceptually to Remix VM for structural validation
- Verified for syntactic correctness and pattern implementation

6.4 Transaction Verification and Testing

After deployment, transaction verification plays a critical role in confirming whether a smart contract behaves as intended. In this project, blockchain explorers and professional

development tools were used to validate execution correctness and transparency.

Etherscan is a blockchain explorer for the Ethereum network and represents one of the most important tools for post-deployment verification. It allows developers and auditors to:

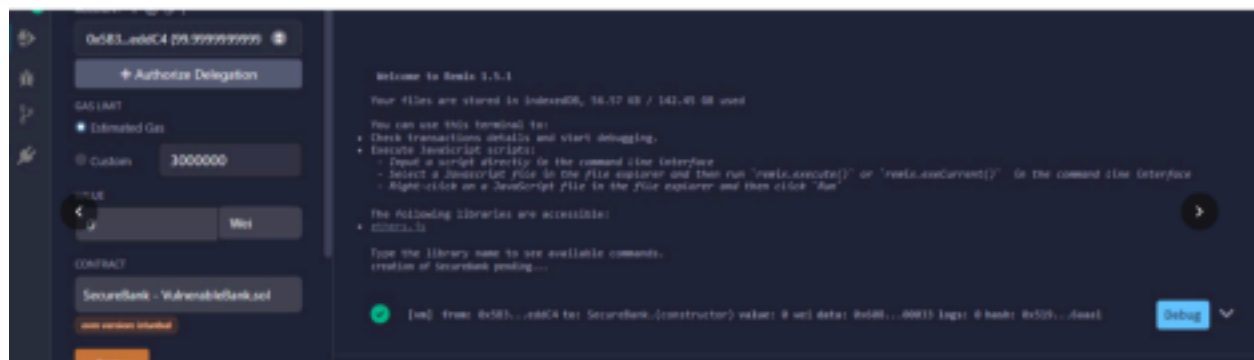
- View and confirm whether a transaction has been successfully executed
- Inspect transaction status (success or failure)
- Analyze gas fees and execution costs
- Verify smart contract code and compare it with deployed bytecode
- Track emitted events and state changes

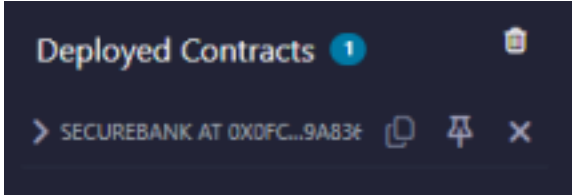
Using Etherscan ensures that every interaction with the smart contract is publicly visible and immutable, reflecting the real-world conditions of Ethereum mainnet. This transparency is essential for detecting unexpected behavior, debugging logic errors, and establishing trust in decentralized applications.

In addition to Etherscan, professional development environments such as **Hardhat** and **Ganache** represent alternative testing approaches. These tools allow developers to run a local Ethereum blockchain, deploy contracts in an isolated environment, simulate transactions, and debug smart contract execution step by step. Hardhat, in particular, provides advanced debugging capabilities, automated testing frameworks, and detailed error tracing.

While Hardhat and Ganache are typically used during earlier development and testing phases, Etherscan is crucial after deployment, as it reflects how the contract behaves in a real or public test network. Together, these tools form a complete verification workflow that reduces deployment risk and improves overall smart contract reliability.

Using Etherscan :





Transaction details for [vm] from: 0x583...ed0C4 to: VulnerableBank.(constructor) value: 0 wei data: 0x08...F0033 logs: 0 hash: 0x575...2eaeec

status: 1 Transaction mined and execution succeed

transaction hash: 0x57553831b25876736689a2675db9934c31bd8638271cF1F519ca5b6d2eaeec

block hash: 0xae0854bFca0f63cebeebfc327bed4f41d3b75d104c7Ffa3351d48b79b92c1a07

block number: 0

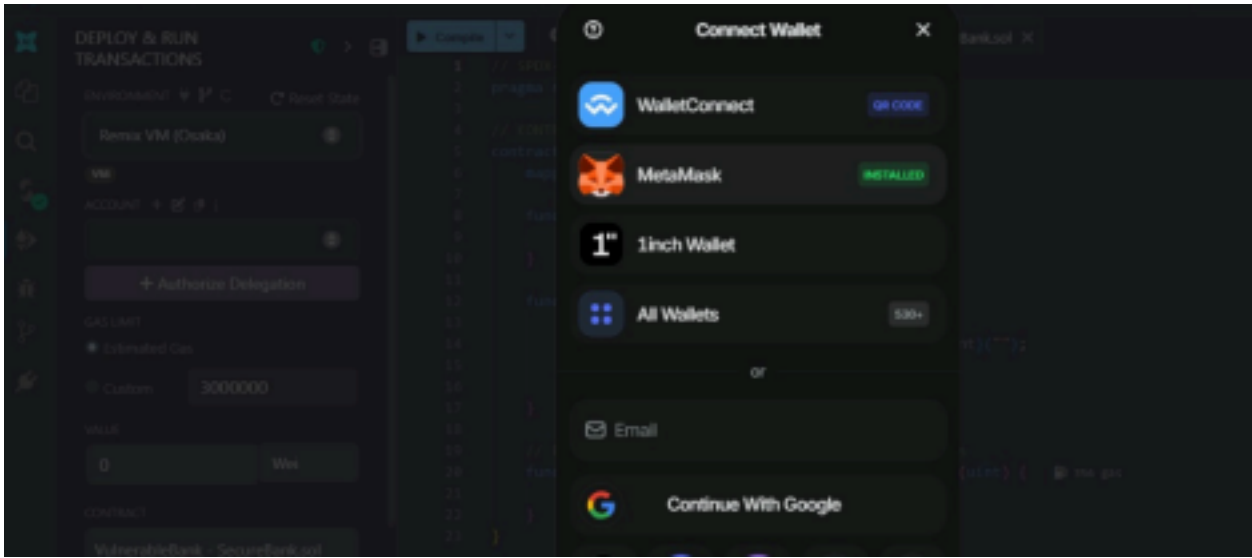
contract address: 0x807F748C41E726C9588488e57fa6129e3b1E99

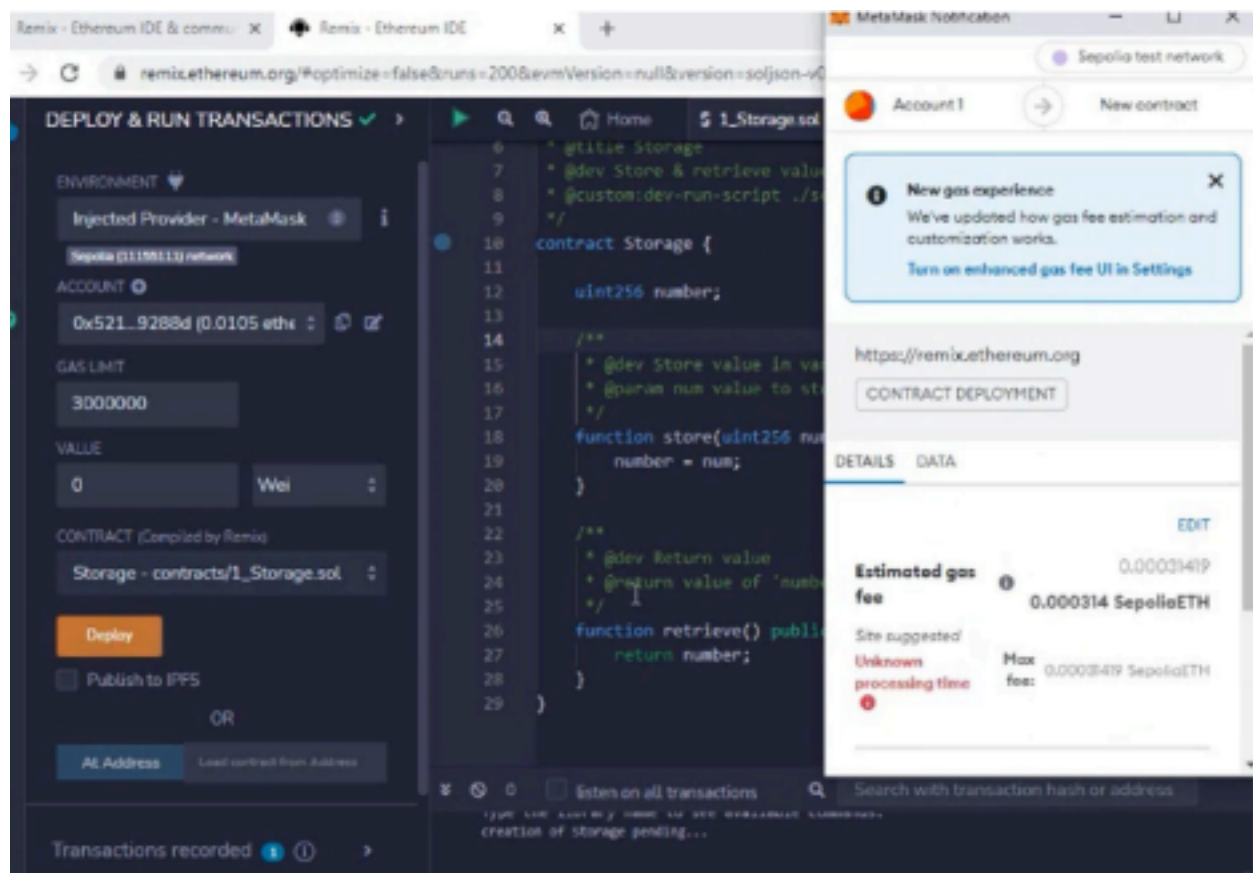
from: 0x58380e6a781c568545dCfc883Fcb875f56bed0C4

Latest 25 from a total of 357 transactions

Transaction Hash	Method	Block	Age	From	To	Amount	Txn Fee
0x80d0ba9b267...	Transfer	10129283	43 hrs ago	0e993a0f36...33293adD9	0x0FC5029C...4Df98A836	0.05 ETH	0.0000013

Using MetaMask :





7. SECURITY EVALUATION AND EFFICIENCY ANALYSIS

The evaluation phase focuses on comparing the behavior of the vulnerable and secure smart contracts. The vulnerable contract demonstrates how improper ordering of operations enables reentrancy attacks, allowing repeated withdrawals before state variables are updated.

In contrast, the secure contract applies the Check-Effects-Interactions (CEI) pattern and reentrancy guard mechanisms. Internal state variables are updated before any external calls are executed, effectively closing the attack window. This approach significantly improves the contract's resilience against malicious interactions.

From an efficiency perspective, while additional security checks introduce minimal computational overhead, the trade-off is justified by the substantial reduction in financial risk. In decentralized systems where contracts manage real economic value, prioritizing security over minor performance gains is essential.

8. DISCUSSION: LINKING THEORY WITH PRACTICE

The practical implementation carried out in this project strongly validates the theoretical

research presented in earlier sections. Vulnerabilities described in academic literature, such as reentrancy, are often perceived as abstract or purely conceptual risks. However, the implementation of a deliberately vulnerable smart contract demonstrates that these weaknesses are concrete, reproducible mechanisms that can be exploited under real execution conditions.

By observing the behavior of the vulnerable contract during compilation, deployment, and transaction execution, it becomes evident how small logical oversights—such as improper ordering of state updates and external calls—can open critical attack windows. This behavior directly mirrors the exploitation pattern observed in the DAO attack, where theoretical flaws in state management resulted in one of the most severe financial losses in blockchain history.

The secure contract further reinforces the connection between theory and practice by showing how well-established mitigation strategies, such as the Check-Effects-Interactions pattern and reentrancy guards, effectively eliminate the identified risks. This comparison highlights that smart contract security is not achieved through complex logic, but through disciplined application of fundamental design principles.

Overall, this discussion emphasizes that theoretical knowledge alone is insufficient unless it is consistently applied during implementation. The transition from theory to practice is where security decisions become irreversible, particularly in immutable blockchain environments. Therefore, understanding vulnerabilities.

9. CONCLUSION AND LESSONS LEARNED

This project demonstrates that smart contract security is not merely a theoretical concern but a practical necessity in blockchain-based financial systems. Through theoretical research, historical analysis, and controlled implementation, the study shows how reentrancy vulnerabilities emerge from subtle logical flaws and how they can lead to severe financial consequences.

The practical component confirms that secure development practices must be integrated throughout the entire lifecycle of a smart contract, from initial design to deployment and verification. Tools such as Remix IDE, MetaMask, and Etherscan are not only development aids but essential components of a secure workflow.

One of the key lessons learned is that immutability amplifies the cost of mistakes. Unlike traditional software systems, errors in smart contracts cannot be easily fixed after deployment. Therefore, developers must adopt a security-first mindset, rely on established patterns, and learn from historical failures such as the DAO attack.

In conclusion, this project reinforces the idea that bridging theory with practice is critical for developing secure decentralized applications. By understanding past vulnerabilities and applying disciplined development methodologies, future blockchain systems can achieve greater reliability, trust, and long-term sustainability.